LeetCode Deep Dive

From First Instinct to Optimal Algorithm: Bipartite Coloring and Debt Settlement in Practice

Week 11 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

The four-step process, once more

1. **First instinct.** Write the algorithm that follows most directly from re-reading the problem statement – usually try-every-possibility.
2. **Measure why it hurts.** Plug the actual constraints into the naive algorithm's complexity and watch it blow up.
3. **Find the structural insight.** What property of the problem removes almost all of that wasted work?
4. **Implement the optimal algorithm**, trace it by hand, then look for programming-level implementation tricks.

This week's twist

Problem 1 (*Is Graph Bipartite?*) gets a clean polynomial optimal algorithm – exactly the pattern from every previous week. Problem 2 (*Optimal Account Balancing*) does **not**: it previews next week's topic, NP-completeness, where step 3 simply has no polynomial answer to find.

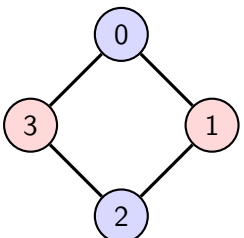
Problem statement

LeetCode 785 – Is Graph Bipartite? (Medium)

You are given an undirected graph on n vertices $0, \ldots, n-1$ as an adjacency list `graph` (no self-loops, no duplicate edges; possibly disconnected). Return `true` iff the vertices can be split into two sets so that every edge has one endpoint in each set.

- Constraints (LeetCode): $1 \leq n \leq 100$.
- This is exactly this week's **bipartiteness** definition – the same property that made the $s \to X \to Y \to t$ matching reduction well-defined in the first place.

Worked example



`graph = [[1,3],[0,2],[1,3],[0,2]]`

- A 4-cycle $0{-}1{-}2{-}3{-}0$: colors $\{0, 2\}$ (blue) vs. $\{1, 3\}$ (red) – every edge crosses colors. **Answer: true.**
- Contrast: a *triangle* $0{-}1{-}2{-}0$ (odd cycle) can never be 2-colored – walking the cycle forces colors $A, B, A, \ldots$ and an odd-length walk returns to a vertex demanding it be both A and B. **Answer: false.**

First instinct: try every 2-coloring

Most direct reading of the problem

"Split vertices into two sets" $\Rightarrow$ try every way of assigning each vertex to set A or set B, and check whether *some* assignment makes every edge cross sets.

1: **for** each of the 2^n assignments $c : V \to \{A, B\}$ **do**
2: **if** every edge (u, v) has $c(u) \neq c(v)$ **then**
3: **return true**
4: **end if**
5: **end for**
6: **return false**

Why this is bad

- Checking one assignment costs $O(E)$; there are 2^n assignments.
- Total: $O(2^n \cdot E)$. At the problem's own bound $n = 100$: $2^{100} \approx 1.3 \times 10^{30}$ – utterly impossible, even though $n = 100$ is a tiny graph.
- The search treats each vertex's color as an **independent** choice. It is not: once one vertex's color is fixed, bipartiteness (if it holds at all) **forces** every neighbor's color.

The smell to notice

Whenever "try all 2^n / $n!$ possibilities" is the first instinct, ask: are these choices really independent, or does fixing one determine the rest?

The structural insight

- Within one connected component, pick any start vertex s and color it A. Every neighbor of s is then **forced** to B (if the graph is bipartite at all); every neighbor of *those* is forced back to A; and so on.
- So there is at most **one** candidate 2-coloring per component, not 2^n – generate it by propagation (BFS/DFS), and check for a conflict (an edge whose two endpoints already share a color) *as you go*.
- This is this week's own 2-coloring characterization of bipartiteness, run forward as an algorithm instead of used as a definition.

Optimal algorithm: BFS 2-coloring

```
 1: function IsBipartite(G = (V, E))
 2:     color[v] ← None for all v
 3:     for each s ∈ V with color[s] = None do
 4:         color[s] ← A; push s onto queue Q
 5:         while Q not empty do
 6:             u ← Q.pop
 7:             for each neighbor v of u do
 8:                 if color[v] = None then
 9:                     color[v] ← opposite(color[u]); push v onto Q
10:                 else if color[v] = color[u] then
11:                     return false                    ▷ conflict: same color, adjacent
12:                 end if
13:             end for
14:         end while
15:     end for
16:     return true
17: end function
```

One BFS pass per component, $O(V + E)$ total.

Trace on the worked example

`graph = [[1,3],[0,2],[1,3],[0,2]]`, BFS from 0.

step	action	queue after	conflict?
1	$color[0] \leftarrow A$, push 0	[0]	–
2	pop 0; neighbors 1, 3 uncolored $\rightarrow color[1], color[3] \leftarrow B$	[1, 3]	no
3	pop 1; neighbor 0 is A ($\neq B$), ok; neighbor 2 uncolored $\rightarrow A$	[3, 2]	no
4	pop 3; neighbor 0 is A ($\neq B$), ok; neighbor 2 already A ($\neq B$), ok	[2]	no
5	pop 2; neighbors 1, 3 both B ($\neq A$), ok	[]	no

Answer

No conflict found across the whole (single) component $\Rightarrow$ **true**, coloring $\{0, 2\}$=A, $\{1, 3\}$=B – matches the diagram.

Complexity: naive vs. optimal

Approach	Time	At $n = 100$
Try all 2^n colorings	$O(2^n E)$	$\sim 10^{30}$ ops – impossible
BFS/DFS propagation	$\boldsymbol{O(V + E)}$	$\leq\sim 5000$ ops – instant

The gap is not "polynomial vs. slightly-worse polynomial" – it is "polynomial vs. impossible," because the naive search never noticed that almost every one of its 2^n choices was already determined by an earlier one.

Python implementation

```python
from collections import deque

def isBipartite(graph):
    n = len(graph)
    color = [0] * n                  # 0 = uncolored, 1 / -1 = the two colors

    for start in range(n):
        if color[start] != 0:
            continue
        color[start] = 1
        queue = deque([start])
        while queue:
            u = queue.popleft()
            for v in graph[u]:
                if color[v] == 0:
                    color[v] = -color[u]         # forced to the opposite color
                    queue.append(v)
                elif color[v] == color[u]:
                    return False                  # same color, adjacent: conflict
    return True
```

Implementation tips & tricks (the programming side)

- **Encode colors as ± 1, not strings/enums.** `-color[u]` flips the color in one machine op; comparing small integers is faster than comparing strings, and `0` doubles cleanly as the "uncolored" sentinel.
- `deque.popleft()`**, not** `list.pop(0)`**.** A plain Python list's `pop(0)` is $O(n)$ (it shifts every remaining element), silently turning a BFS you believe is $O(V + E)$ into $O(V^2)$ in practice.
- **Return** `False` **the instant a conflict appears.** Don't finish coloring the whole graph first and check afterward – exiting early avoids wasted work exactly like Week 3's Union-Find returning at the first cycle-closing edge.
- **Loop over all vertices as potential BFS starts.** The graph may be disconnected; forgetting the outer `for start in range(n)` loop silently ignores every component other than the first.

Problem statement

LeetCode 465 – Optimal Account Balancing (Hard)

A list of transactions $[from_i, to_i, amount_i]$ records that person $from_i$ gave person to_i exactly $amount_i$ dollars. Return the **minimum number of transactions** required to settle all debts among the group.

- Only each person's **net balance** matters – who owes whom directly is irrelevant, only the final settlement count.
- This is a flow-*shaped* problem (supplies and demands, like this week's circulations-with-demands), but the objective – minimizing the transaction **count** – is not one max-flow answers. Keep that tension in mind.

Worked example

`transactions = [[0,1,10],[2,0,5]]`

	person 0	person 1	person 2
after $[0, 1, 10]$ (0 pays 1)	-10	$+10$	0
after $[2, 0, 5]$ (2 pays 0)	$-10 + 5 = -5$	$+10$	-5

Nonzero balances: $0 : -5$, $1 : +10$, $2 : -5$.

A settlement in 2 transactions

0 pays 1 exactly \$5 (zeroes out 0, leaves 1 at $+5$); 2 pays 1 exactly \$5 (zeroes out both). **Answer: 2** – matches LeetCode's own Example 1.

First instinct: try every settlement order

Most direct reading of the problem

"Find the minimum number of transactions" $\Rightarrow$ try every way of pairing up nonzero balances and transferring money between them, and take the best.

- Naively: at each step, pick *any* two nonzero balances of opposite sign and transfer the smaller amount between them; branch over **every** such pair, recurse, count transactions, minimize over all branches.
- With k nonzero balances, the number of distinct ways to fully pair/group them grows combinatorially – for $k = 12$, well past 10^8 branches with no pruning.

Why this is bad

- Unlike Problem 1, there is **no known polynomial algorithm** for this objective: deciding whether k nonzero balances can be settled in exactly $k - g$ transactions (for g independent zero-sum groups) is equivalent to partitioning a multiset into zero-sum subsets – a problem in the same NP-hard family as *Partition* and *Subset Sum*.
- So "why is this bad" is not just "the constant is too big" – it is "no algorithm known to anyone runs in polynomial time on this exact objective." This is next week's topic previewed early.

The honest complexity story

LeetCode bounds `transactions.length` ≤ 8, so at most 16 people and usually far fewer nonzero balances after netting – small enough that a well-pruned exponential search is fast in practice, even though no polynomial algorithm exists.

The structural insight (pruning, not a polynomial bound)

- **WLOG reduction:** in *some* optimal solution, the person holding the *first* nonzero balance transacts with one specific other person. So at each recursion level, only try pairing balance[0] with each of the other nonzero balances – branching factor $k - 1$ instead of an unstructured search over all pairings.
- **Free pruning:** if balance[0] happens to already be 0 (because an earlier transfer zeroed it exactly), skip straight to the next nonzero balance *without* counting a transaction – this automatically detects and exploits any zero-sum subgroup that can settle independently.
- Still exponential worst case – but these two rules are exactly what makes the search fast enough for LeetCode's $k \leq 16$.

Optimal (best-known) algorithm: pruned backtracking

```
 1: function SETTLE(balances[])
 2:     skip leading zero entries (advance index i while balances[i] = 0)
 3:     if i ≥ length of balances then return 0
 4:     end if
 5:     best ← ∞
 6:     for each j > i with balances[j] opposite sign to balances[i] do
 7:         balances[j] ← balances[j] + balances[i]; save old balances[i]
 8:         balances[i] ← 0
 9:         best ← min(best, 1 + SETTLE(balances[]))
10:         undo: restore balances[i] and balances[j]
11:     end for
12:     return best
13: end function
```

Exponential worst case; fast in practice because of the pruning above.

Trace on the worked example

Nonzero balances (order found): $[-5, +10, -5]$ (person 0, 1, 2).

call	state	choice
$\textsc{Settle}([-5, 10, -5])$	$i = 0$, *balances*$[0] = -5$	try $j = 1$: $10 - 5 = 5 \to [0, 5, -5]$
$\textsc{Settle}([0, 5, -5])$	$i = 1$ (*balances*$[0]$ now 0, skip free)	try $j = 2$: $-5 + 5 = 0 \to [0, 0, 0]$
$\textsc{Settle}([0, 0, 0])$	all zero	**return** 0

Answer

Unwinding: $1 + (1 + 0) = 2$ transactions – $0 \to 1$ (\$5), $2 \to 1$ (\$5). Matches the hand-computed answer.

Python implementation (part 1: net balances)

```
def minTransfers(transactions):
    net = {}
    for u, v, amount in transactions:
        net[u] = net.get(u, 0) - amount
        net[v] = net.get(v, 0) + amount
    balances = [b for b in net.values() if b != 0]
    # settle(...) is defined next, then called as: return settle(0)
```

Only *nonzero* net balances ever enter the search – everyone else is already settled and contributes zero transactions.

Python implementation (part 2: pruned backtracking)

```
    def settle(i):
        while i < len(balances) and balances[i] == 0:
            i += 1                                  # free: already-zero prefix
        if i == len(balances):
            return 0
        best = float('inf')
        for j in range(i + 1, len(balances)):
            if balances[j] * balances[i] < 0:       # opposite signs only
                balances[j] += balances[i]
                old = balances[i]
                balances[i] = 0
                best = min(best, 1 + settle(i + 1))
                balances[i] = old
                balances[j] -= balances[i]
        return best

    return settle(0)
```

Implementation tips & tricks (the programming side)

- **Work in integer cents, not float dollars.** Summing floating-point amounts can leave a balance at `-4.999999999` instead of exactly `-5`, so the "skip if balance $==$ 0" pruning silently breaks. LeetCode's inputs are integers – keep every intermediate value an `int`.
- **Filter zero balances before recursing.** Building `balances` only from `net.values() if b != 0` shrinks the search space immediately – people who already balance out never enter the backtracking at all.
- **Mutate-and-undo beats copying the list.** Restoring `balances[i]`/`balances[j]` after the recursive call avoids allocating a new list at every one of the (many) recursive calls – a real constant-factor win when the search tree is already exponential.

Summary: two problems, one contrast

1. **Is Graph Bipartite?** follows this course's usual arc: a naive 2^n search turns out to be full of redundant, forced choices, and exploiting that gives a clean $O(V + E)$ polynomial algorithm – BFS 2-coloring, the same idea underlying this week's matching reduction.
2. **Optimal Account Balancing** breaks that arc on purpose: minimizing the transaction *count* is equivalent to a zero-sum partitioning problem with no known polynomial algorithm. The best we can do is prune an exponential search (WLOG pairing + free zero-skipping) and rely on the input being small.
3. That contrast *is* the lesson: flow-shaped problems are not automatically easy – max-flow makes *matching size* and *cut value* easy, but a different objective on the same kind of data (transaction *count*) can land you in NP-hard territory instead.

Keep practicing

`LEETCODE.md` lists four more Week 11 problems (*Possible Bipartition*, *Maximum Number of Accepted Invitations*, *Minimum Number of Vertices to Reach All Nodes*, *Course Schedule II*) – and next week's Intractability lecture will explain *why* Problem 2 above resisted a clean polynomial algorithm.